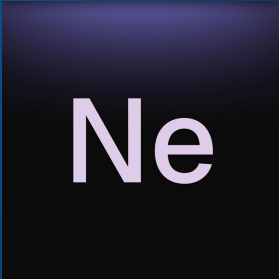


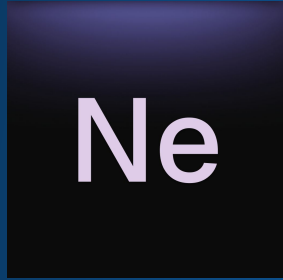
Low Level C++ Theory & Applications

Amlal El Mahrouss

The logo consists of a dark blue square with a subtle gradient and a slight shadow effect. Inside the square, the letters "Ne" are written in a white, sans-serif font.

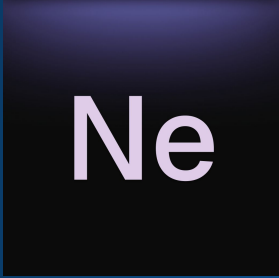
Ne

I'm Amlal from Ne.app!



Email: amlal@ne-app.eu

I'm Amlal from Ne.app!

The logo for Ne, consisting of the letters 'Ne' in a white, sans-serif font, set against a dark purple square background with a subtle gradient.

Ne

These slides will serve as supporting content for the talk, It is not excluded that I will take questions from time to time.

As I am for the talk to be as interactive as possible.

Purpose of the talk:

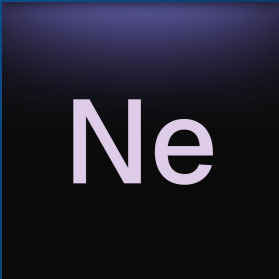
→ In this talk we're going to layout techniques and decisions to take in order to successfully develop a low-level system in modern C++.

The following standards are involved:

→ C++11 and later. This is because we'd want more reach over the talk than if this was restricted only to C++17 and later.

On References:

- Unreference claims are not included in this talk for the sake of accuracy.
- P.S: These slides are available on GitHub. [publications-and-more/talk](#).
- Thank you.

The logo consists of a dark blue square with a subtle gradient. Inside the square, the letters "Ne" are written in a white, sans-serif font. The "N" is slightly larger than the "e".

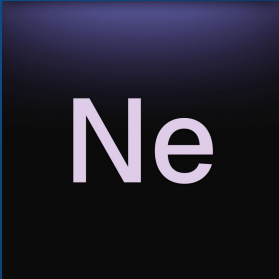
Ne

I: 'Core' Principles Covered.

Amlal El Mahrouss

Observation: What we will cover in this talk.

- Common RAII Patterns.
- When to use RAII?
- List of Several RAII Containers.
- Applications and Examples.
- Error Handling and Graceful Exits.

The logo for the Ne programming language, featuring the letters "Ne" in a white, sans-serif font centered on a dark purple square background with a subtle gradient.

Ne

II: Several Common RAII Patterns.

Amlal El Mahrouss

When to use RAII?

When using non-freestanding C++ RAII is mostly a ‘no-brainer’:

→ `std::unique_ptr`.

→ `std::shared_ptr`.

However, one may judge to roll its own container when deemed necessary, such as Open C++ Libraries, ‘`tracked_ptr`’, or Boost’s custom expansions of shared pointers operations such as `delete_ptr`.

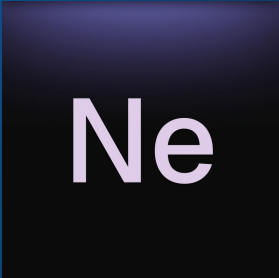
When to use RAII?

An example:

Ant resource handling in HAL.dll and minoskrnl.exe:

It's architecture makes use of the RAII pattern to manage resources that are deemed to be eligible for such operations.

Another example, file descriptors are when used by a routine are automatically freed after the routine finishes running.

The logo consists of the letters 'Ne' in a white, sans-serif font, centered within a dark blue square. The square has a subtle gradient and a slight shadow effect, giving it a three-dimensional appearance as if it's floating or attached to the slide.

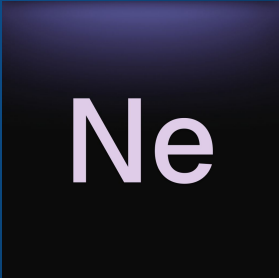
When to use RAII?

Example: A pool allocator in a graphics engine or high-throughput system.

This is a non-trivial operation, where the complexity of the algorithm may be exponential here. Thus per RAII, we will need to perform caching and implement a pool allocation algorithm.

This helps for the following two concepts:

- Linearity of the search.
- Searching time per algorithm complexity.

The logo for Ne, featuring the letters 'Ne' in a white, sans-serif font, centered on a dark blue square background with a subtle gradient.

III: A List of Several RAII Containers and Patterns.

Amlal El Mahrouss

The OwnPtr, shared_ptr, and auto_ptr.

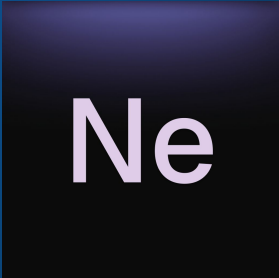
OwnPtr, A type introduced in WebKit, back when C++0x wasn't formalized yet.

Another example: Ne.app Ant and Ne.app NeSystem have their own RAII primitives.

And so does many C++ software...

Even some C based software does (thinking about the GCC extensions for that purpose)

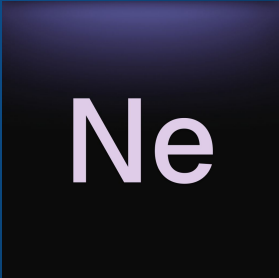
Source: [Implementing RAII in pure C? - Stack Overflow](#)

The logo for Ne, consisting of the letters 'Ne' in a white, sans-serif font, centered on a dark blue square background with a subtle gradient.

Design Matters!

After shipping and maintaining several software projects, I've come to this conclusion, and RAII containers are a prime example of this. What was once started with `auto_ptr`, had to be scaled to several refactorors (unique, shared pointers) post C++11 due to the demand of more demanding containers.

I would love to talk more about how could one would design such software in the long-term, but I'm afraid It's not the scope of the talk.

The logo consists of a dark blue square with a subtle gradient. Inside the square, the letters "Ne" are written in a white, sans-serif font. The "N" is slightly larger and more prominent than the "e".

Ne

IV: Applications and Examples Low Level C++.

Amlal El Mahrouss

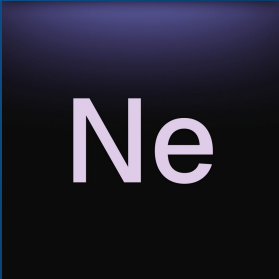
Example one: A Telnet Server

\$ telnet telnet.example.com (Made up server host)

Now... One shall handle, the TUI, resources allocation, logging, etc...

What happens if the resource is busy? What happens if the NIC is being plugged off? Or the router crashes?

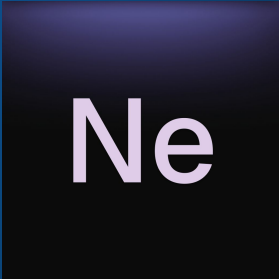
That's why RAII is crucial, it helps handles those cases!

A dark blue square with a gradient, containing the letters "Ne" in a white, sans-serif font.

Example one: A Telnet Server

\$ telnet telnet.example.com (Made up server host)

One may wrap the resources into a `Ref` container and pointers into a `RefPtr` container. Thus giving us the ability to handle the telnet program resources more effectively.

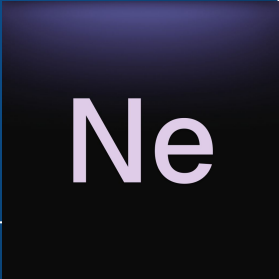
The logo consists of a dark blue square with a subtle gradient and a slight shadow. Inside the square, the letters "Ne" are written in a white, sans-serif font.

Example one: A Telnet Server

```
$ telnet telnet.example.com (Made up server host)
```

One may wrap the resources into a ``Ref`` container and pointers into a ``RefPtr`` container. Thus giving us the ability to handle the telnet program resources more effectively.

However, abuse of RAII concepts and patterns, will lead to issues with the program down the line. As it will become increasingly difficult

The logo consists of the letters 'Ne' in a white, sans-serif font, centered within a dark blue square with a subtle gradient and a slight shadow effect.

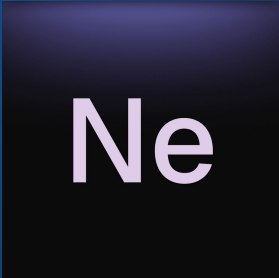
Example: System level allocators, and abstracting it into a RAII container.

The following snippet is written in C:

```
VoidPtr x = MmAllocHeap(sz, flags, ...);
```

This compiles in C, but could be improved for ‘higher level applications’*

[1] An application that does not require manual management
Of the process’ memory.

The logo for Ne, consisting of the letters 'Ne' in a white, sans-serif font, centered on a dark purple square background with a subtle gradient.

Example: System level allocators, and abstracting it into a RAII container.

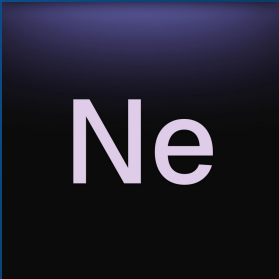


```
VoidPtr x = MmAllocHeap(sz, flags, ...);  
if (x != nullptr) {  
    /// Turn 'x' in a pool or whatever...  
}
```

Ne

Example: System level allocators, and abstracting it into a RAII container.

Gets turned into this...

The logo for Ne, consisting of the letters 'Ne' in a white, sans-serif font, centered on a dark blue square background with a subtle gradient and a slight shadow effect.

Ne

Example: System level allocators, and abstracting it into a RAII container.

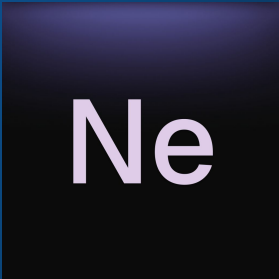


```
/// Now the x is freed when doing function unwinding on the stack.  
OwnPtr<VoidPtr> x = MmAllocHeap(sz, flags, ...);  
if (x) {  
    /// Turn 'x' in a pool or whatever...  
}
```

Ne

Example: System level allocators, and abstracting it into a RAII container.

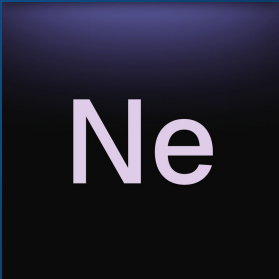
This applies well for this example, under a specific caveat: The resource scope and lifetime has to be determined and predictable.

The logo for the Ne programming language, featuring the letters "Ne" in a white, sans-serif font centered within a dark blue square with a subtle gradient and a slight drop shadow.

Ne

Example: System level allocators, and abstracting it into a RAII container.

Now several notes...

The logo for Ne, consisting of the letters 'Ne' in a white, sans-serif font, centered within a dark blue square with a subtle gradient and a slight shadow effect.

Ne

Example: System level allocators, and abstracting it into a RAII container.

Note: One may provide a deleter predicate on the container initialization for allocators that have non-trivial behavior and are not compatible with the standard OS allocator. E.g: COM, Qt...

[1]

<https://stackoverflow.com/questions/9308862/memory-management-in-com>

V: Error Handling and Graceful Exits.

Amlal El Mahrouss

Per Jason Turner's Talk on Library Design...

One shall think about error handling in the infancy of the program's development...

This should be present in the early days of the library.

[1]

https://www.youtube.com/watch?v=zL-vn_pGGgY

V/VI: Conclusion.

Amlal El Mahrouss

Thank you for listening!

Amlal El Mahrouss

Thank you for listening!

There is slides available on my site:
amlal.nekernel.org (or the mirror pub.ne-app.eu)

Questions?

Amlal El Mahrouss